Hashing

Hash : Chop and mix

The symbol #

Hash Store : Older form of supermarkets

Hashing

Hash Function

A hash function of *h* is given as H=h(m), where

h : Hash function,

m : variable length message,

H : fixed length hash value of the message.

Also known as:

- The message digest,
- The fingerprint or,
- The digital fingerprint

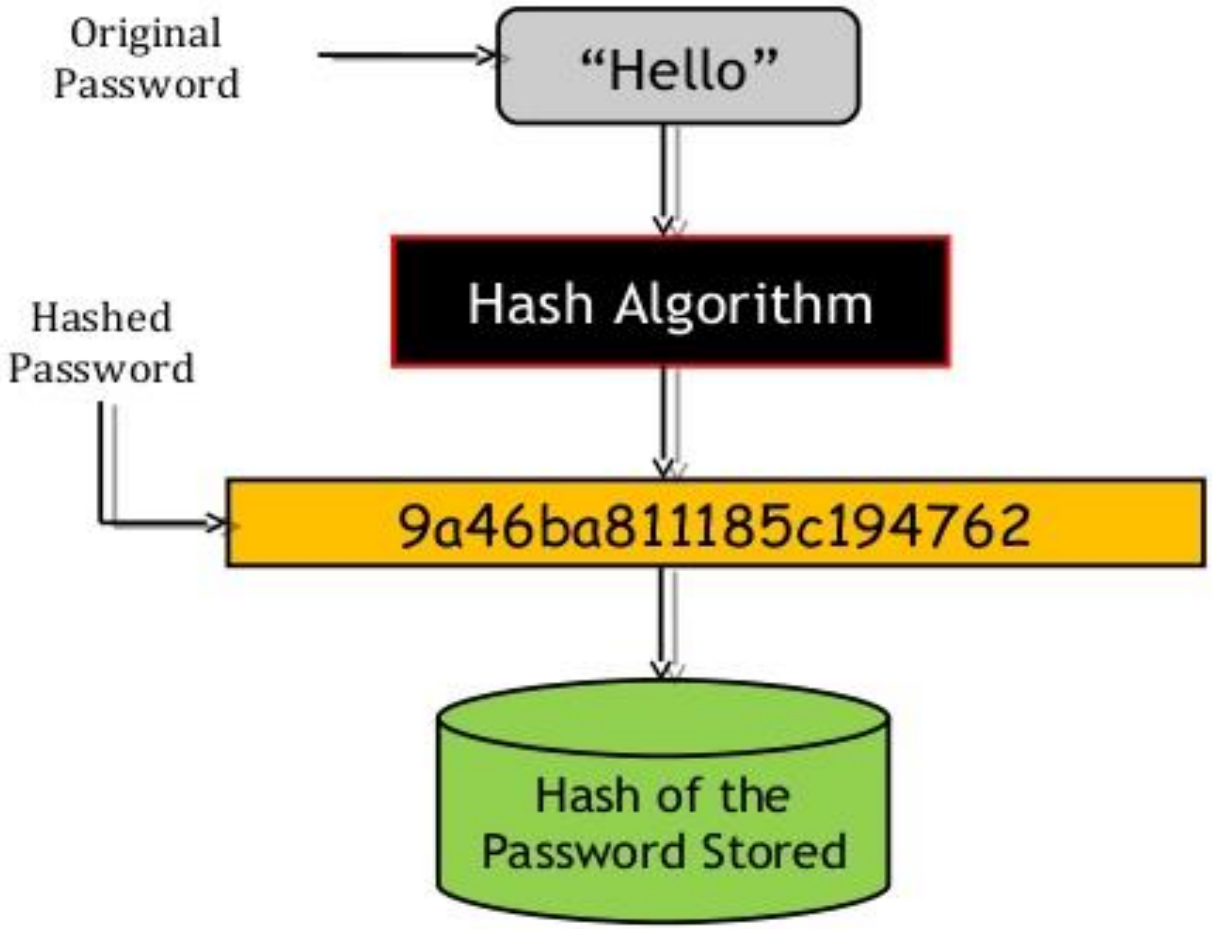
$\mathcal{H}=h(m)$

- The hash value (H) should be calculated relatively easily for any given message (m).
- h(m) should be a one-way function (computationally intractable).
- There should be only one H value for any given m (collision free).

$\mathcal{H}=h(m)$

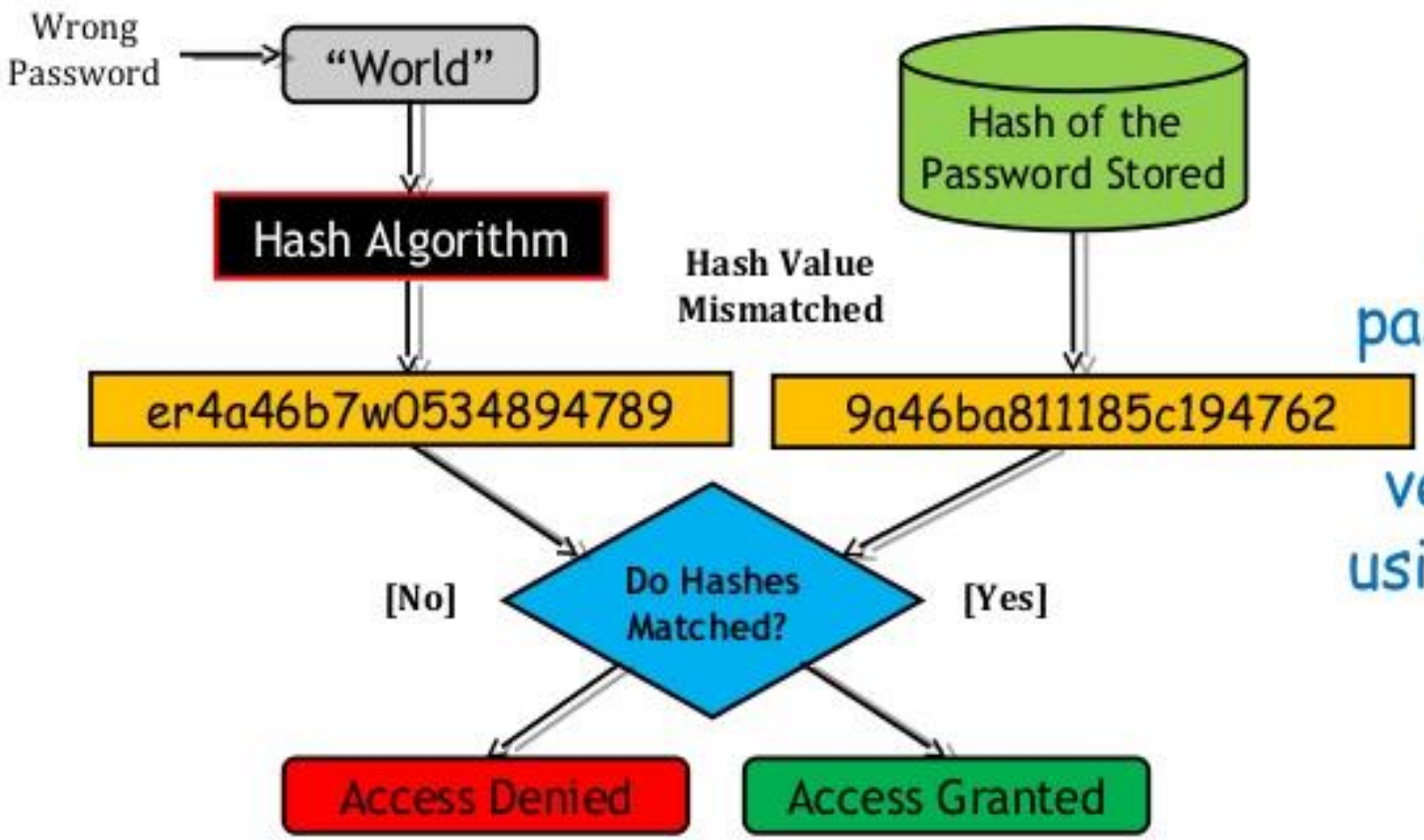
- SHA-1 (Secure Hash Algorithm-1) was announced as a standard hashing algorithm FIPS PUB 180-1 by National Institute for Standards and Technology (NIST) of USA in 17.4.1995 and it produced 160 bit H value.
- NIST announced FIPS PUB 180-2 in August 1 2002. With this update, SHA-256, SHA-384 & SHA-512 surfaced.
- H values from SHA family are 160, 256, 384 & 512 bits respectively, in FIPS PUB 180-2.

Application: Hashing Password



How password is stored using hash

Applications of Hash



How password is verified using hash

Hash Tables

- In file and database operations , hashing is used for performing insertions, deletions and searches in constant average time (O(1)).
- Hashing is useful as an indexing method and as a search method in symbol tables.
- Symbol table - repository of all information within a compiler. All parts of a compiler communicate thruough this table and access the data-symbols.
- Stores info about scope and binding information of various entities such as variable,label and function names, classes, objects, etc.

Searching Records in Files

- Find items with keys matching a given search key
  - Given a structure A, containing n keys, and a search key k, find the element i such as k=A[i]
  - As in the cases of sorting or searching, a key could be part of a large record.

example of a record

Key	**other data**

Searching Records in Files

- Key is used in computing the location value for the item.
  - Key could be an *integer*, a *string*, etc
  - e.g. a name or Id that is a part of a large employee record
- The items that are stored in the hash table T are indexed by values from *0* to *TableSize – 1*.
- Each key is mapped into some number in the range 0 to *TableSize – 1.*
- Mapping is performed using a *hash function*.

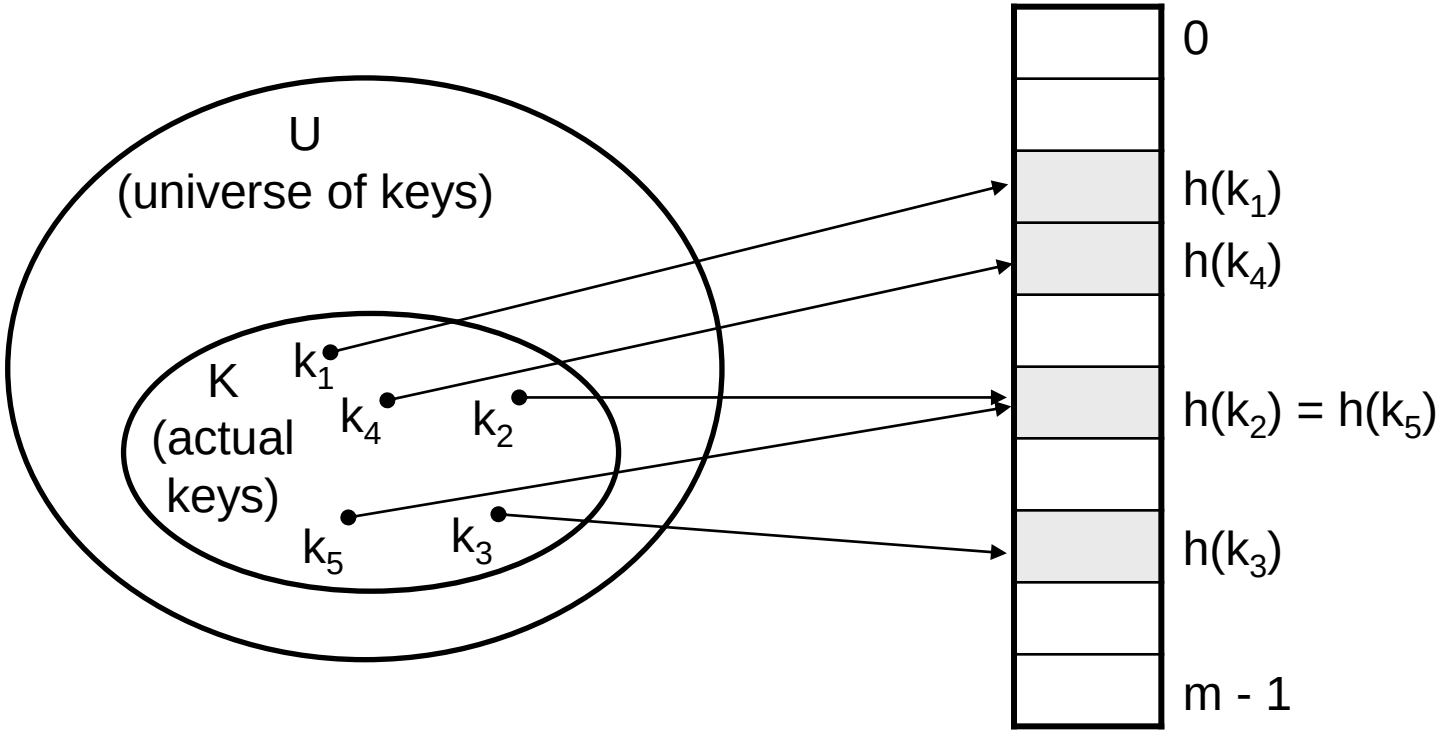
Forming Hash Tables

- Use a function h to compute the slot(Address) for each key k
- Store the element in slot h(k)
- A hash function h transforms a key into an index in a hash table T[0…m-1]:

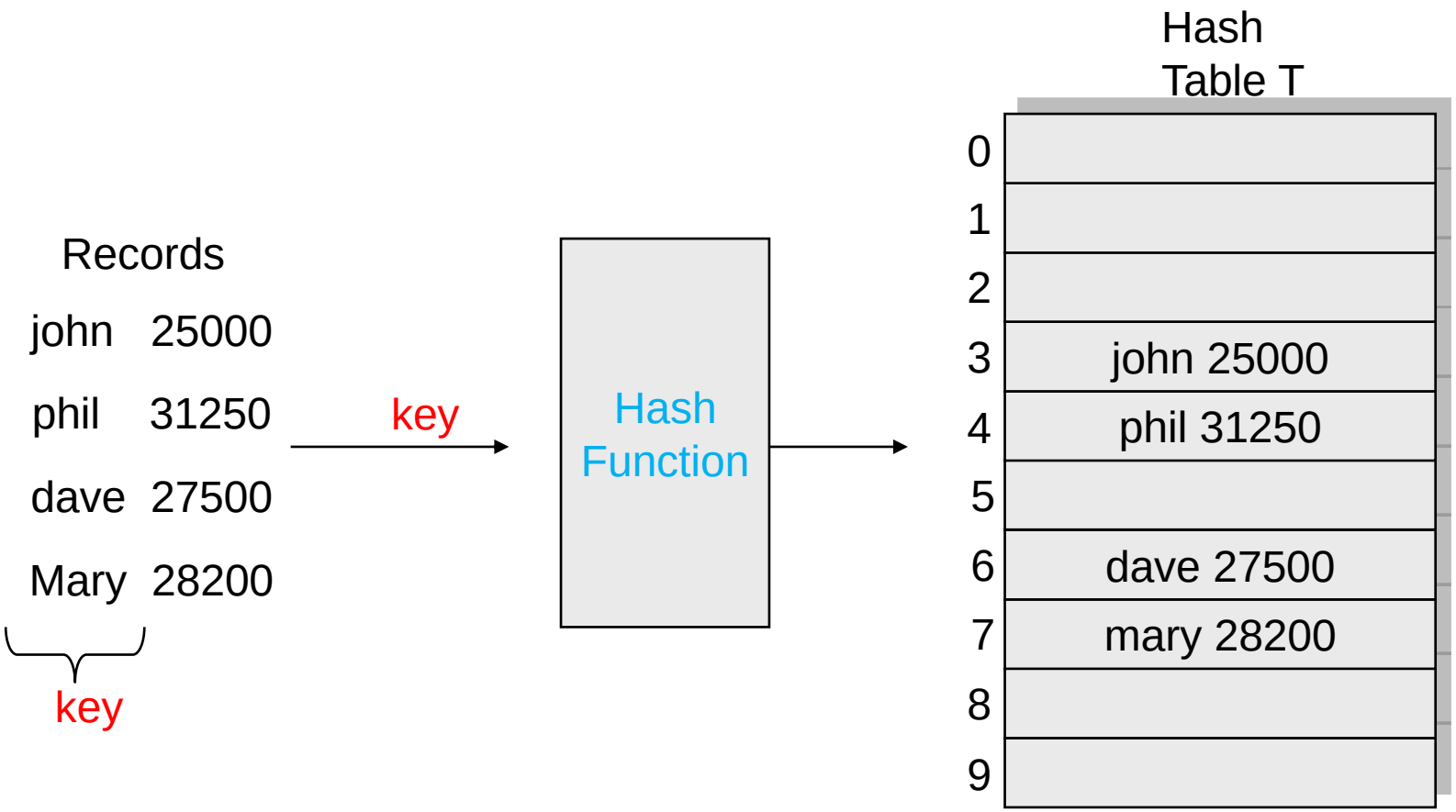
  $$h : k \rightarrow \{0, 1, \ldots, m - 1\}$$

- We say that h hashes key k to slot h(k)
- Advantages:
  - Reduce the range of array indices handled
  - Storage is also reduced

Example: HASH TABLES



Example Hash Function and Hash Table

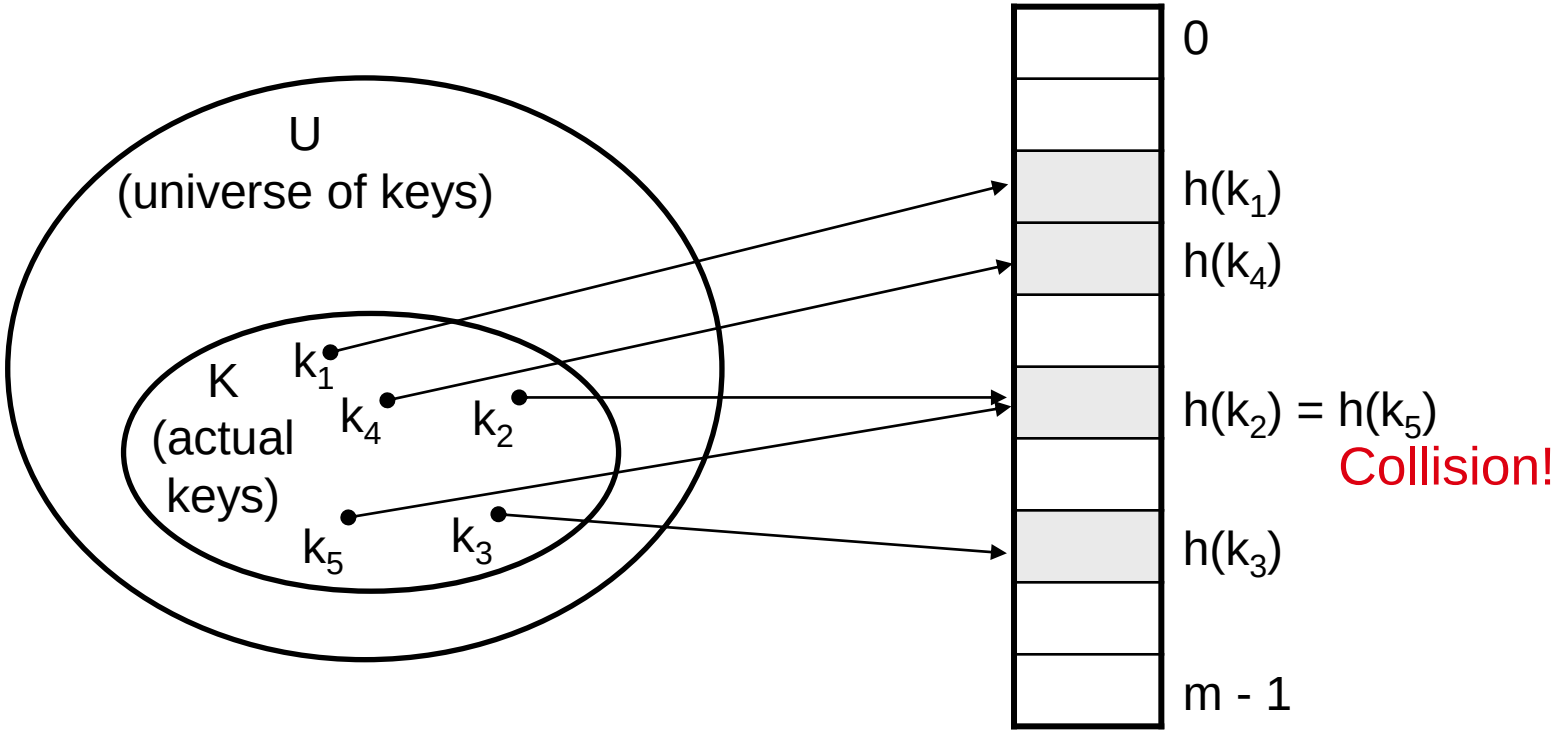


Hashing: Problems

Problems:

- Keys may not be numeric.
- Number of possible keys is much larger than the space available in the table.
- Different keys may map into the same location
  - Hash function is not one-to-one
  - In this case, the performance of hashing will suffer dramatically.

Do you see any problem ?



Collisions

When the function assigns the same address to >1 records collisions occur.

Example:

- - Hash table size 11
- - Hash function: key mod hash size

23 mod 11 =1 , 18 mod 11= 7, 29 mod 11=7,…

So, the new positions in the hash table are:

Key	23	18	29	28	39	13	16	42	17
Position	1	7	7	6	6	2	5	9	6

collisions: positions 6, 7.

Hash Functions

- A hash function transforms a key into a table address
- What makes a good hash function?

  (1) Easy to compute

  (2) Approximates a random function: for every input, every output is equally likely :simple uniform hashing
- In practice, it is very hard to satisfy the simple uniform hashing property
  - i.e., we don't know in advance the probability distribution that keys are drawn from

Some Hash Function Methods

- Division Method : Key mod m:
  - m is the size of the table, better if it is prime.
- Folding:
  - 123|456|789: add them and take mod.
- Truncation:
  - 123456789 map to a table of 1000 addresses by picking any 3 digits of the key.
- Mid-Square:
  - Square the key and then truncate to desired range

……..

- A good hash function is designed to distribute the keys roughly evenly into the available positions within the array (or hash table).

The Division Method

- Map a key k into one of the m slots by taking the remainder of k divided by m

$$h(k) = k \bmod m$$

- This gives the indexes in the range 0 to m-1 so the hash table should be of size m

Example**:**Suppose we have a table of size 1000.

A good choice could be the nearest prime number m=1009.

If k=5127 then the corresponding hash value is

$h(k)$=5127 mod 1009=82 [5127= 5*1009 + 82]

<u>Properties:</u>

- Fast, requires only one operation but , certain values of m are bad. Ex : Powers of 2.

The Folding Method

- The key K is partitioned into a number of parts ,each of which has the same length as the required address with the possible exception of the last part .
- The parts are then added together , ignoring the final carry, to form an address.

<u>Example:</u> If key=456 942 781 is to be transformed into a three digit address.

P1=456, P2=942, P3=781

h(key)= (Last three digits (456+942+ 781))

=079.

The Mid- Square Method

- The key K is multiplied by itself and the address is obtained by selecting an appropriate number of digits from inside the square.
- The number of digits selected depends on the size of the table.
- <u>Example:</u> If key=123456 and m=1000:

$(123456)^2=15241383936$

- Since a three-digit address is required, positions 5 to 7 could be chosen, giving the address 138.

Hashing a String Key

- Table size [0..99].Assign numerical values to the symbols that can be used to form a key :
  - A..Z ---> 1,2, ...26
  - 0..9 ----> 27,...36
- Key: CS1 →3+19+28 (concat) = 31,928

  $(31{,}928)^2$ = 1019397184 - 10 digits
- Extract middle 2 digits (5th and 6th) as requested table size is 0..99.

  Get 39.

→ H(CS1) = 39.

Hashing a String Key

- Sum up ASCII (or Unicode) values for characters in string s
  - ASCII value for "A" =65,...,Z=90. sum will be in range 650-900 for 10 upper-case letters.

  EX: Hash code for string "abracadabra" is

  ASCII("a")+ ASCII("b")+...+ ASCII("a") = X

  →X can be used as a hash value or one of the previous methods can be used to convert X into the desired range.

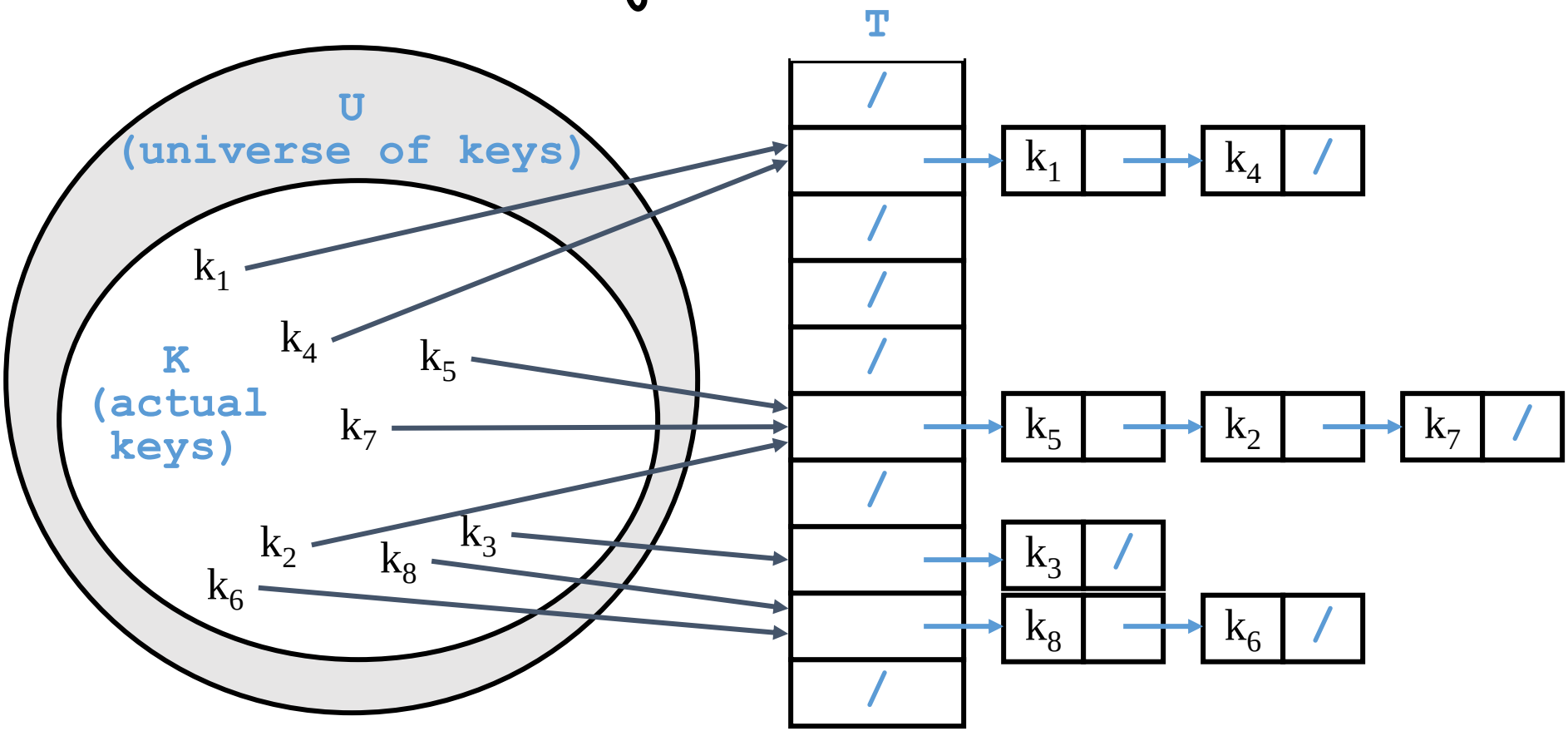
Has the order of chars in string any effect ?

Handling Collisions

- There are several methods for dealing with collision problem:
  - Chaining
  - Open addressing
    - Linear probing
    - Quadratic probing
    - Double hashing

Handling Collisions Using Chaining

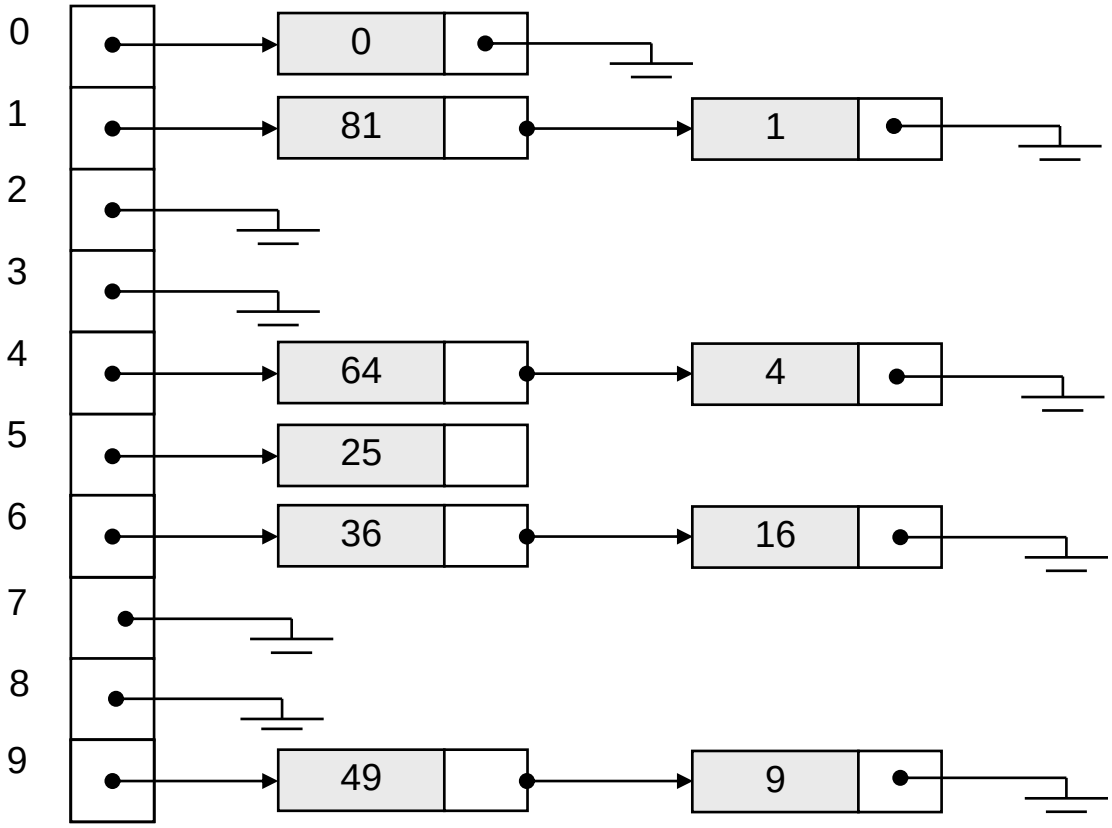
- Chaining puts elements that hash to the same slot in a linked list.
- Slot j contains a pointer to the head of the list of all elements that hash to j



Chaining : Example

Keys: 0, 1, 4, 9, 16, 25, 36, 49, 64, 81,m=10 addresses

hash(key) = key % 10.



Chaining : Table Properties

- Choosing the size of hash table:
  - Small enough not to waste space
  - Large enough such that collision lists remain short
- How should we keep the lists, ordered or not?
  - Not ordered, Why ?
    - Insertion is fast
    - Can easily delete the most recently inserted elements

Chaining: Hash Table Operations

- Initialization: All entries are set to NULL
- Insertion:
  - Locate the table cell(bucket), using hash function.
  - Insert the collided item to the front of the list.
- Find(Search):
  - locate the cell ,using hash function.
  - sequentially search the linked list for that cell.
- Deletion:
  - Locate the cell ,using hash function.
  - Delete the item from the linked list.

Chaining : Deletion in Hash Tables

CHAINED-HASH-DELETE(T, x)

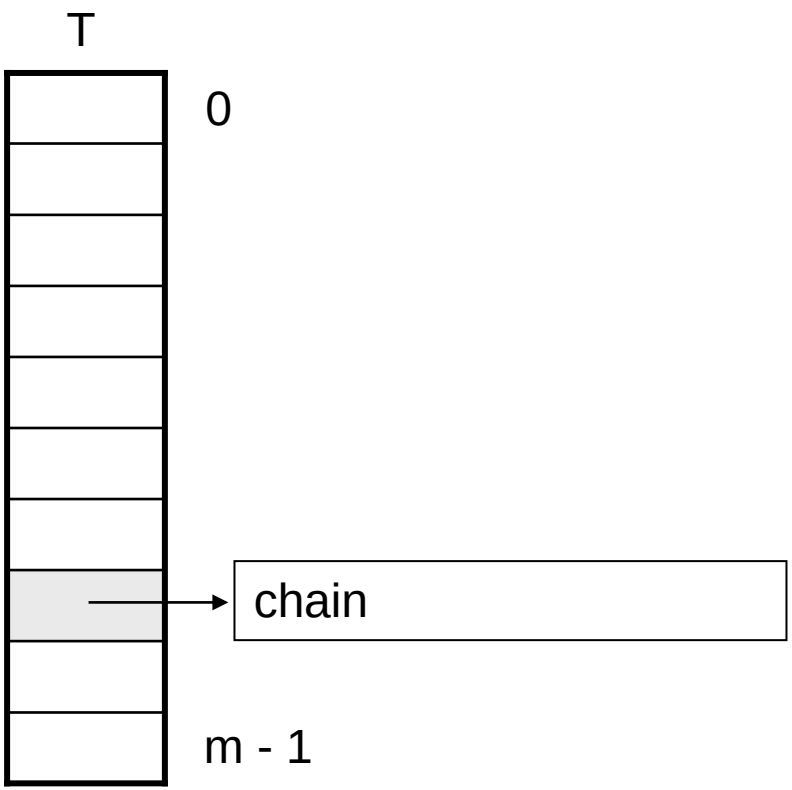
//Delete x from the head of its list

$T[h(key[x])] \leftarrow NIL$

- Need to find the element to be deleted.
- Worst-case running time
  - Deletion time depends on searching the corresponding list

Analysis of Hashing with Chaining:

- **Cost** :How long does it take to search for an element with a given key?
- Worst case:
  - All n keys hash to the same slot
  - Worst-case time to search is $\Theta(n)$, plus time to compute the hash function



Handling Collisions : Open Addressing

- Separate chaining has the disadvantage of using linked lists.
  - Requires the implementation of a second data structure.
- In an open addressing hashing system, all the data go inside the table.
  - Thus, a bigger table is needed.
  - If a collision occurs, alternative cells are tried until an empty cell is found.
- There are three common open addressing collision resolution strategies:
  - Linear Probing
  - Quadratic probing
  - Double hashing

Linear probing: Inserting a key

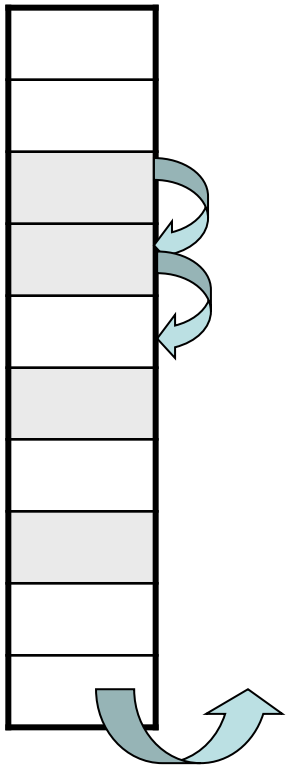
- Idea: when there is a collision, check the next available position in the table (i.e., probe ahead)

$$h(k,i) = (h_1(k) + i) \bmod m$$

$$i=0,1,2,...$$

- First slot probed: $h_1(k)$
- Second slot probed: $h_1(k) + 1$
- Third slot probed: $h_1(k)+2$, and so on

probe sequence: [h1(k), h1(k)+1 , h1(k)+2 , ...]

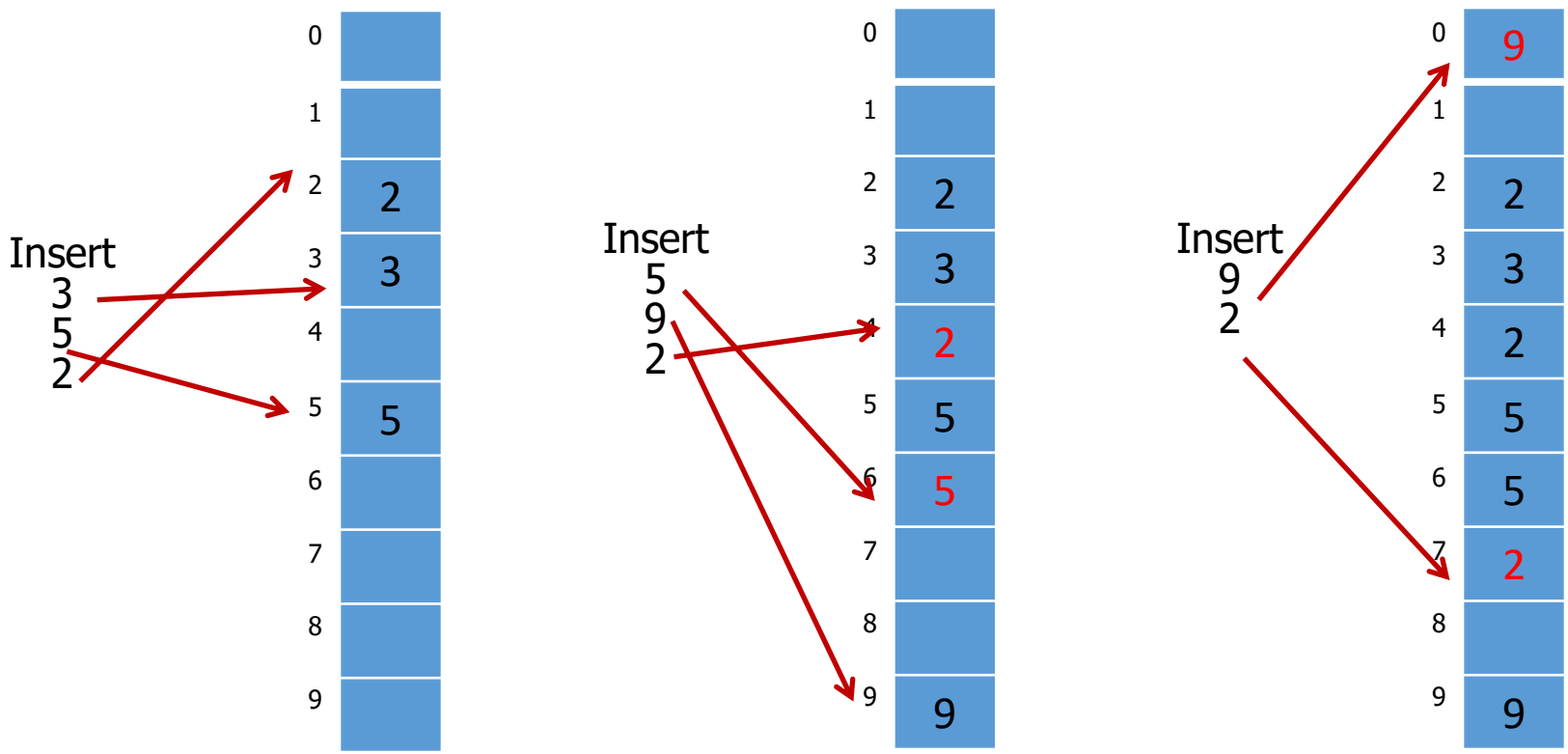


wrap around

Linear Probing: Example

Assume Table size m= 10. Hash function is hash(x) = x mod m.

Values to be inserted : 3,5,2,5,9,2,9,2



Collision resolution by linear probing

Quadratic Probing

- Quadratic Probing eliminates clustering problem of linear probing.
- Collision function is quadratic.
  - The popular choice is : $f(i) = i^2$.

  → i^{th} probe = $(h(k) + i^2) \bmod m$
- If the hash function evaluates to h and the cell h is occupied, we try cells $h + 1^2$, $h+2^2$, $h + 3^2$…
  - i.e. It examines cells 1,4,9 and so on away from the original probe.

A quadratic probing hash table after each insertion

```
hash ( 89, 10 ) = 9
hash ( 18, 10 ) = 8
hash ( 49, 10 ) = 9
hash ( 58, 10 ) = 8
hash (  9, 10 ) = 9
```

	After insert 89	*After insert 18*	*After insert 49*	*After insert 58*	*After insert 9*
0			49	49	49
1					
2				58	58
3					9
4					
5					
6					
7					
8		18	18	18	18
9	89	89	89	89	89

Problem of Quadratic Probing

- We may not be sure that we will probe all locations in the table.
- There is no guarantee to find an empty cell if table is more than half full.
- If the hash table size is not prime this problem will be much severe.

Handling Collisions with Rehashing

- Double Hashing is one of the best methods for dealing with collisions.
  - If the slot is full, then a second hash function is calculated and combined with the first hash function.
- Rehashing: If collision occurs; Try H_1, H_2, …, H_m in sequence. Here each H_i is a different hash function.

We are going to consider Double Hashing.

Double Hashing

Idea: Spread out the search for an empty slot by using a second hash function

1. Use one hash function to determine the first slot:

   Initial probe: $h_1(k)$

2. We apply a second hash function to k and probe at a distance: $h_2(k)$, $2*h_2(k)$, … and so on.

- Second probe is offset by $h_2(k)$ **mod** m, so on ...

$$h(k,i) = (h_1(k) + i\, h_2(k)) \bmod m$$

$$i=0,1,\ldots$$

- Disadvantage: harder to delete an element

Double Hashing Example

Insert keys : 76,93,40,47,10,55
Hash functions : $h_1(k) = k \bmod 7$, $h_2(k) = 5 - (k \bmod 5)$

	76	93	40	47	10	55
0						
1				47	47	47
2		93	93	93	93	93
3					10	10
4						55
5			40	40	40	40
6	76	76	76	76	76	76
Probes	1	1	1	2	1	2

Problems of Collision Resolution

- Collision resolution is a costly process.With many collisions, hashing becomes inefficient.
- A good hash function should minimize collisions and provide a balanced distribution of records to addresses or buckets.
- But this is not guaranteed in many cases and frequent file reorganizations may be required.

Performance of Hashing

- A good hash function should be simple and should be computed quickly.Performance degrades if hash function is slow.
- If there is no collision, the time complexity is fixed:

  $T = O(1)$

  →Does not depend on the size!

  →Typically 1-2 attempts will be enough to locate the key value.

Static and Dynamic Hashing

- The hashing methods we have considered so far are "static" i.e. The maximum number of addresses or buckets are predefined.
- In static hashing, function *h* maps search-key values to this fixed set of bucket addresses.
  - If initial number of buckets is too small, performance will degrade due to too much overflows.
  - If there are many deletions, space will be wasted.
- Dynamic hashing does not impose this restriction.The address space grows dynamically in response to increasing needs.

(We shall not consider this topic here.)

Hashing Applications

- Compilers use hash tables to implement the *symbol table*.
- Game programs use hash tables to keep track of positions it has encountered (*transposition table*)
- A hash table can be used for on-line spelling checkers.If misspelling detection (rather than correction) is important, an entire dictionary can be hashed and words checked in constant time
- Hash functions can be used to quickly check for inequality.If two elements hash to different values they must be different.
- Cryptography, digital signing, e-signatures
- File and database indexing and organization.

Hashing in Cryptography

- Another area where hashing is used extensively is cryptography.
- Cryptographic Hashing is a method of cryptography that converts any form of data into a unique string of text.
- Any piece of data can be hashed, no matter its size or type.
- A hash is designed to act as a one-way function: you can put data into a hashing algorithm and get a unique string. But you cannot decipher the data it represents.
- A common application area is password protection.